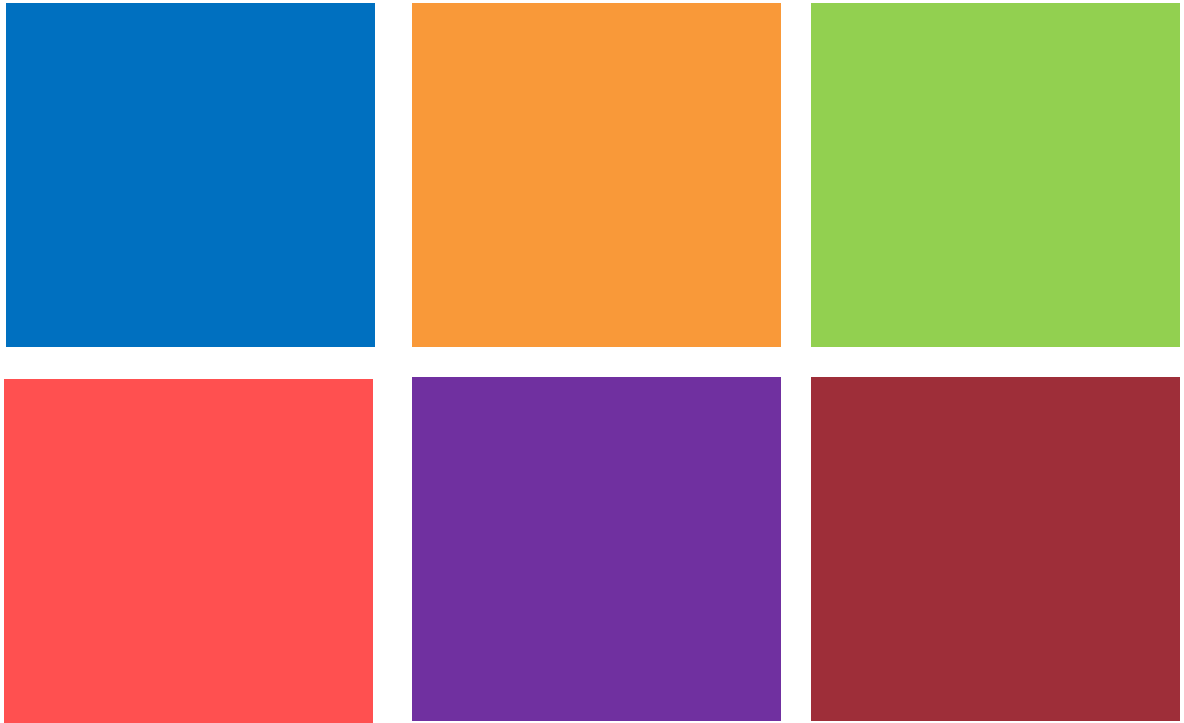
BBM 105 – SOFTWARE ENGINEERING

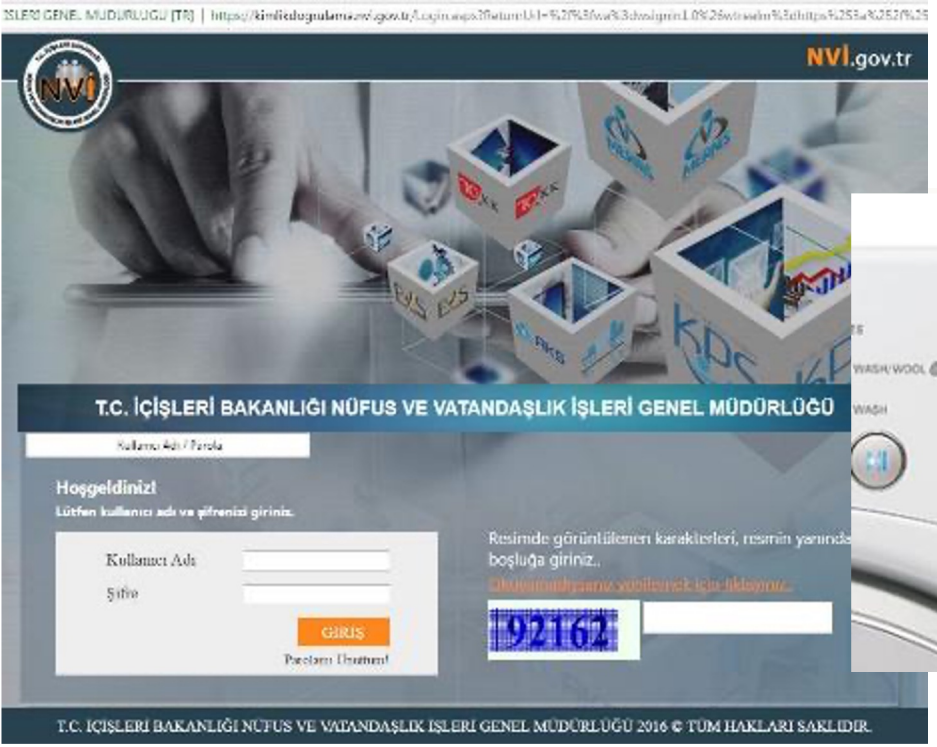
(from Book on Software Engineering by Ian Sommerville)



Software in Modern World

- We can’t run the modern world without software.
  - National infrastructures and utilities are controlled by computer-based systems
  - Most electrical products include a computer and controlling software
  - Industrial manufacturing and distribution is completely computerized, as is the financial system.
  - Entertainment, including the music industry, computer games, and film and television, is software intensive

Software in Modern World: Examples



Challenges

- Characteristics of software:
  - Intangible (abstract)
  - Changeable
- Characteristics of software development:
  - Human-intensive
  - Multi-disciplinary

Software Failures – Main Reasons

- ***Increasing demands***
  - Systems have to be built and delivered more quickly;
  - Larger, even more complex systems are required;
  - Systems have to have new capabilities that were previously thought to be impossible.
  - Existing software engineering methods cannot cope and new software engineering techniques have to be developed to meet new these new demands.

- ***Low expectations***
  - It is relatively easy to write computer programs without using software engineering methods and techniques.
  - Many companies do not use software engineering methods.
  - Consequently, their software is often more expensive and less reliable than it should be.

We need better software engineering education and training to address this problem.

Software Failures – Expectations!

Software Engineering is NOT Programming!

Programming Vs. Engineering

Programming	Software Engineering
Personal activity (instrument)	Team activity (orchestra)
One aspect of software development	Large systems must be developed similar to other engineering practices
Concerned about accomplishing the objective of the program itself	Concerned about the entire solution, its feasibility, and future use

History of Software Engineering

- The notion of ‘software engineering’ was first proposed in 1968 at a conference held to discuss what was then called the ‘software crisis’ (Naur and Randell, 1969).
  - It became clear that individual approaches to program development did not scale up to large and complex software systems. These were unreliable, cost more than expected, and were delivered late.

- Throughout the 1970s and 1980s, a variety of new software engineering techniques and methods were developed, such as structured programming, information hiding and object-oriented development. Tools and standard notations were developed and are now extensively used.

Professional Software Development

- Professional software, intended for use by someone apart from its developer, is usually developed by teams rather than individuals. It is maintained and changed throughout its life.

- Software engineering is intended to support professional software development, rather than individual programming. It includes techniques that support program specification, design, and evolution.
  - None of which are normally relevant for personal software development.

Professional Software Development

- Many people think that software is simply another word for computer programs. However, software is not just the programs themselves but also all associated documentation and configuration data that is required to make these programs operate correctly.
  - A professionally developed software system usually consists of system documentation, which describes the structure of the system; user documentation, which explains how to use the system, and web- sites for users to download recent product information.

- This is one of the important differences between professional and amateur software development.

Software Products

- Software engineers are concerned with developing software products (i.e., software which can be sold to a customer).

- There are two kinds of software products:
  - *Generic products*
  - *Customized (or bespoke) products*

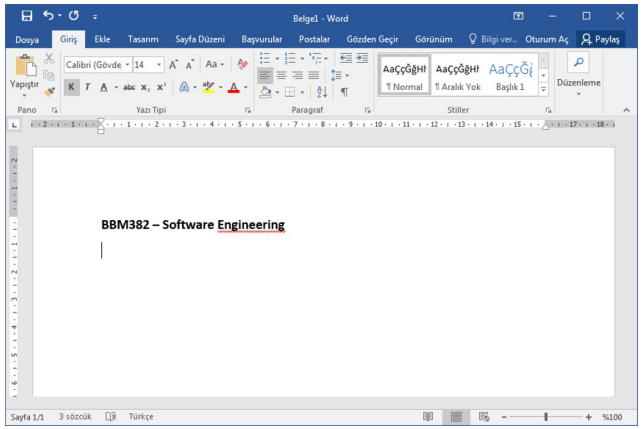
Kinds of Software Products

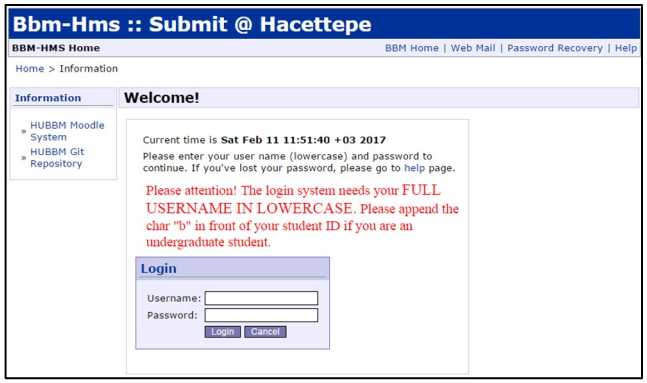
- *Generic products* These are stand-alone systems that are produced by a development organization and sold on the open market to any customer who is able to buy them.
  - Examples include software for PCs such as databases, word processors, drawing packages, and project-management tools. It also includes some specific purpose applications such as library information systems, accounting systems, or systems for maintaining dental records.

- *Customized (or bespoke) products* These are systems that are commissioned by a particular customer. A software contractor develops the software especially for that customer.
  - Examples of this type of software include control systems for electronic devices, systems written to support a particular business process, and air traffic control systems.

Kinds of Software Products

- An important difference between these types of software is
  - In *generic products*, the organization that develops the software controls the software specification (*e.g. MS Word*)
  - For *custom products*, the specification is usually developed and controlled by the organization that is buying the software (*e.g. Submit system*)





- The mix of two is also possible!
  - Systems are being built with a generic product as a base, which is then adapted to suit the requirements of a customer.
    - Enterprise Resource Planning (ERP) systems, such as the SAP system, are the best examples.

Quality

- When we talk about the quality of professional software, we have to take into account that the software is used and changed by people apart from its developers.
  - Quality is therefore not just concerned with what the software does; it has to include the software's behavior while it is executing and the structure and organization of the system programs and associated documentation.

- This is so-called quality or non-functional software attributes.
  - Examples of these attributes are the *software's response time to a user query* and the *understandability of the program code*.
  - The specific set of attributes that you might expect from a software system obviously depends on its application domain. Therefore, a banking system must be secure, an interactive game must be responsive, a telephone switching system must be reliable, and so on.

Essential Attributes of Good Software

Product characteristics	Description
Maintainability	Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.
Dependability and security	Software dependability includes a range of characteristics including reliability, security, and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
Efficiency	Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilization, etc.
Acceptability	Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable, and compatible with other systems that they use.

Software Engineering

- Software engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification through to maintaining the system after it has gone into use.
  - *Engineering discipline - Engineers make things work. They apply theories, methods, and tools where these are appropriate. However, they use them selectively and always try to discover solutions to problems even when there are no applicable theories and methods. Engineers also recognize that they must work to organizational and financial constraints so they look for solutions within these constraints.*
  - *All aspects of software production - Software engineering is not just concerned with the technical processes of software development. It also includes activities such as software project management and the development of tools, methods, and theories to support software production.*

Engineering Discipline

- Engineering is about getting repeatable results of the required quality within the schedule and budget.
  - This often involves making compromises—engineers cannot be perfectionists.
  - People writing programs for themselves, however, can spend as much time as they wish on program development.

- In general, software engineers adopt a systematic and organized approach to their work, as this is often the most effective way to produce high-quality software.
  - However, engineering is all about selecting the most appropriate method for a set of circumstances so a more creative, less formal approach to development may be effective in some circumstances.
  - Less formal development is particularly appropriate for the development of web-based systems, which requires a blend of software and graphical design skills.

Software Engineering is Important

Software engineering is important for two reasons:

- More and more, individuals and society rely on advanced software systems. We need to be able to produce reliable and trustworthy systems economically and quickly.

- It is usually cheaper, in the long run, to use software engineering methods and techniques for software systems rather than just write the programs as if it was a personal programming project.
  - For most types of systems, the majority of costs are the costs of changing the software after it has gone into use.

Software Engineering vs. Computer Science

- Computer science is concerned with the theories and methods that underlie computers and software systems, whereas software engineering is concerned with the practical problems of producing software.
- Some knowledge of computer science is essential for software engineers in the same way that some knowledge of physics is essential for electrical engineers.
- Computer science theory, however, is often most applicable to relatively small programs. Elegant theories of computer science cannot always be applied to large, complex problems that require a software solution.

Types of Software Applications

- There are many different types of application including:
  - *Stand-alone applications*
  - *Interactive transaction-based applications*
  - *Embedded control systems*
  - *Batch processing systems*
  - *Entertainment systems*
  - *Systems for modeling and simulation*
  - *Data collection systems*
  - *Systems of systems*

Software Engineering Diversity

- You use different software engineering techniques for each type of system because the software has quite different characteristics.
  - For example, an embedded control system in an automobile is safety-critical and is burned into ROM when installed in the vehicle. It is therefore very expensive to change. Such a system needs very extensive verification and validation so that the chances of having to recall cars after sale to fix software problems are minimized. User interaction is minimal (or perhaps nonexistent) so there is no need to use a development process that relies on user interface prototyping.
  - For a web-based system, an approach based on iterative development and delivery may be appropriate, with the system being composed of reusable components. However, such an approach may be impractical for a system of systems, where detailed specifications of the system interactions have to be specified in advance so that each system can be separately developed.

Software Engineering Diversity

- Nevertheless, there are software engineering fundamentals that apply to all types of software systems:
  - They should be developed using a managed and understood development process. The organization developing the software should plan the development process and have clear ideas of what will be produced and when it will be completed. Of course, different processes are used for different types of software.
  - Dependability and performance are important for all types of systems. Software should behave as expected, without failures and should be available for use when it is required. It should be safe in its operation and, as far as possible, should be secure against external attack. The system should perform efficiently and should not waste resources.

Software Process

The systematic approach that is used in software engineering is sometimes called a software process. A software process is a sequence of activities that leads to the production of a software product:

- Software specification, where customers and engineers define the software that is to be produced and the constraints on its operation.
- Software development, where the software is designed and programmed.
- Software validation, where the software is checked to ensure that it is what the customer requires.
- Software evolution, where the software is modified to reflect changing customer and market requirements.

- There are also supporting process activities such as documentation and software configuration management

Software Process

- A structured set of activities required to develop a software system.
  - Development of software from scratch in a standard programming language
  - Development by extending and modifying existing systems or by configuring and integrating generic (off-the-shelf) software or system components

An anology:

Suffle receipe (process definition) & Suffle cooking (porcess enactment)

Keto Cheese Souffle Recipe

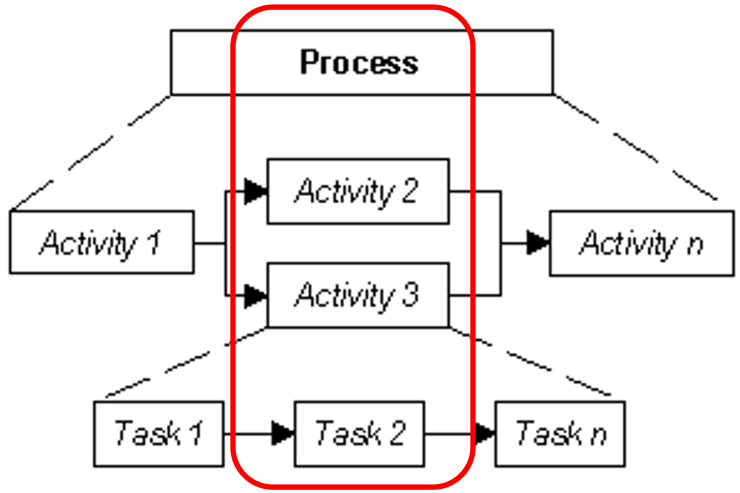
Ingredients:

1 tbsp + 4 tbsp butter	salt + pepper
1 tbsp coconut flour	1 drop tobasco
3 tbsp heavy cream	1 sprinkle nutmeg
2 eggs	1 oz. cheddar cheese

Instructions:

1. Preheat oven to 375 degrees.
2. Use 1 tbsp of butter to coat souffle mold and aluminum collar.
3. Heat butter in sauce pan on medium heat.
4. Add coconut flour and blend with spatula to form a roux. Cook until foamy and frothy, then remove from heat.
5. Once the roux stops foaming, add the heavy cream. Beat with a wisk until the sauce thickens, approximately 1 minute.
6. Separate egg yolks from egg whites. Place egg whites in a large mixing bowl and set to the side.
7. Add egg yolks to white sauce and beat with wisk. Stir in salt, pepper, tobasco and nutmeg. Set to the side.
8. Beat egg whites with a blender. When it starts to foam, add a pinch of salt. Continue beating egg whites until you get a fluffy, slightly stiff consistency. (They should have a sheen to them).
9. Stir a heaping spoon full of the egg whites to the white sauce.
10. Take the white sauce and pour it into the side of the bowl of egg whites. Fold the white sauce into the egg whites, adding the cheddar cheese as you go.
11. Pour souffle mixture into the mold.
12. Place collar around the mold.
13. Bake for 45 minutes.

noexcusegirl.com



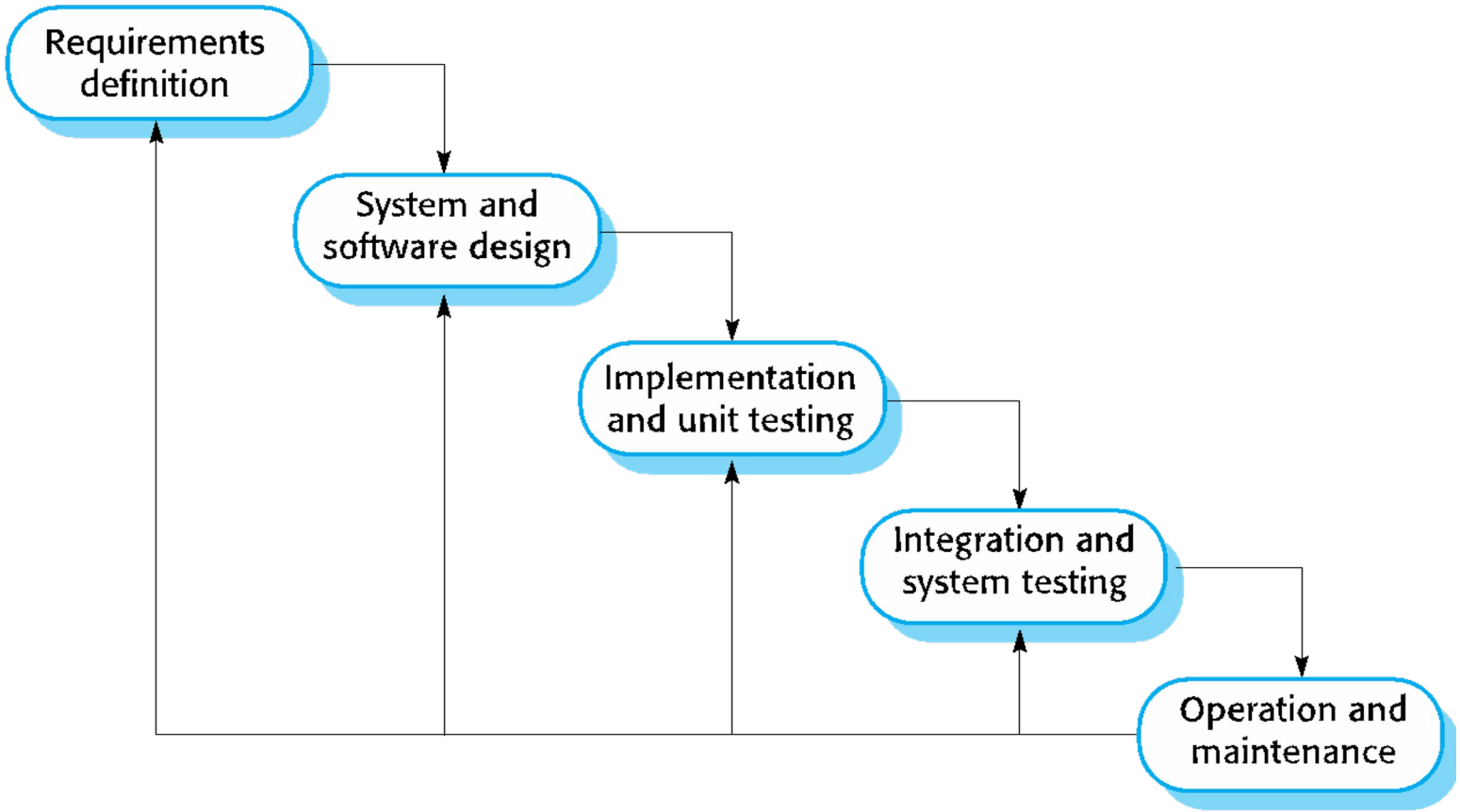
Ref. http://www.chambers.com.au/glossary/activity.php

Plan-driven and agile processes

- **Plan-driven processes** are processes where all of the process activities are planned in advance and progress is measured against this plan.
- **In agile processes**, planning is incremental and it is easier to change the process to reflect changing customer requirements.

  - As Boehm and Turner (2003) discuss, each approach is suitable for different types of software.
  - In practice, most practical processes include elements of both plan-driven and agile approaches.
  - There are no right or wrong software processes – they are tailored according to specific needs of software development.

Example: The Waterfall Model (Plan-Driven)



Because of the cascade from one phase to another, this model is known as the 'waterfall model.' In a plan-driven process—in principle, you must plan and schedule all of the process activities before starting work on them.